

**UNIwersYTET RZESZOWSKI**  
**Kolegium Nauk Przyrodniczych**



Dominik Machnik

Nr albumu: 117814

Sztuczna Inteligencja

## **Przewidywanie Decyzji Odnośnie Udzielenia Pożyczki**

Informatyka II stopień 2024/25

Prowadzący:

dr inż. Jacek Bartman

Rzeszów, 2024

## Spis treści

|     |                                                |    |
|-----|------------------------------------------------|----|
| 1.  | Wstęp .....                                    | 3  |
| 1.1 | Charakterystyka Problemu .....                 | 3  |
| 2.  | Opis Danych.....                               | 4  |
| 3.  | Wstępne przetwarzanie danych.....              | 5  |
| 3.1 | Wczytanie i podstawowe operacje .....          | 5  |
| 3.2 | Obsługa brakujących wartości.....              | 5  |
| 3.3 | Kodowanie zmiennych kategoriycznych .....      | 6  |
| 3.4 | Normalizacja zmiennych numerycznych .....      | 7  |
| 4.  | Projekt Sieci .....                            | 8  |
| 4.1 | Przygotowanie danych do sieci neuronowej ..... | 8  |
| 4.2 | Architektura sieci.....                        | 8  |
| 4.3 | Przebieg uczenia sieci.....                    | 9  |
| 5.  | Analiza rezultatów i wnioski .....             | 11 |
| 5.1 | Miary jakości.....                             | 11 |
| 5.2 | Wnioski .....                                  | 13 |

# 1. Wstęp

## 1.1 Charakterystyka Problemu

Projekt dotyczy zbudowania systemu decyzyjnego, który automatycznie ocenia, czy dany klient banku powinien otrzymać kredyt hipoteczny. W instytucjach finansowych proces zatwierdzania kredytów odgrywa kluczową rolę w zarządzaniu ryzykiem i rentownością. Automatyzacja tej decyzji:

- zwiększa efektywność operacyjną,
- redukuje subiektywność decyzji,
- umożliwia natychmiastowe podejmowanie decyzji w systemach online,
- może ograniczyć ryzyko kredytowe poprzez dokładniejszą segmentację klientów.

Jest to przykład binarnych decyzji kredytowych, które są powszechnie stosowane w sektorze finansowym w ramach tzw. Credit Scoringu. Model ma za zadanie przewidywać wartość zmiennej docelowej **Loan\_Status**, przyjmującej wartości:

- 1 – wniosek kredytowy zaakceptowany,
- 0 – wniosek odrzucony.

## 2. Opis Danych

Zbiór zawiera 614 instancji (wniosków kredytowych), opisanych 13 atrybutami. Dane pochodzą z wniosków kredytowych składanych w instytucji finansowej i służą do predykcji decyzji kredytowych (Loan\_Status).

Charakterystyka atrybutów:

- **Loan\_ID** – Unikalny identyfikator wniosku kredytowego.
- **Gender** – Płeć wnioskodawcy: Male (mężczyzna), Female (kobieta).
- **Married** – Stan cywilny: Yes (żonaty/zamężna), No (niezamężny/singiel).
- **Dependents** – Liczba osób na utrzymaniu wnioskodawcy: 0, 1, 2, 3+ (3 lub więcej).
- **Education** – Wykształcenie: Graduate (wyższe), Not Graduate (bez wyższego).
- **Self\_Employed** – Czy samozatrudniony: Yes (tak), No (nie).
- **ApplicantIncome** – Miesięczny dochód wnioskodawcy.  
Typ: Liczba całkowita (np. 5849).
- **CoapplicantIncome** – Miesięczny dochód współ-wnioskodawcy.  
Typ: Liczba rzeczywista (np. 0, 1508.0).
- **LoanAmount** – Kwota kredytu.
- **Loan\_Amount\_Term** – Okres kredytowania (w dniach)  
Typowe wartości: 360 (12 miesięcy), 180 (6 miesięcy), 480 (16 miesięcy).
- **Credit\_History** – Historia kredytowa: 1: Dobra (brak zaległości), 0: Zła (problemy z spłatą).
- **Property\_Area** – Miejsce zamieszkania kredytobiorcy: Urban (miasto), Semiurban (przedmieścia), Rural (wieś).
- **Loan\_Status** (Zmienna docelowa) – Decyzja kredytowa:  
Y: Kredyt przyznany, N: Kredyt odrzucony.

## 3. Wstępne przetwarzanie danych

### 3.1 Wczytanie i podstawowe operacje

- **Wczytanie danych:**

Dane są ładowane z pliku CSV do tabeli data z zachowaniem oryginalnych nazw kolumn (VariableNamingRule="preserve").

```
data = readtable("train.csv", VariableNamingRule="preserve");
```

- **Usunięcie kolumny ID:**

Kolumna Loan\_ID jest usuwana, ponieważ nie niesie wartości predykcyjnej.

```
data.Loan_ID = [];
```

### 3.2 Obsługa brakujących wartości

- **Analiza braków:**

Dla każdej kolumny obliczany jest odsetek brakujących wartości (missingRatio).

- **Strategie uzupełniania:**

- Jeśli missingRatio > 5% → **uzupełnij braki:**

- *Kolumny numeryczne:* medianą (z pominięciem NaN).
- *Kolumny tekstowe:* modą (najczęstsza wartość).

- Jeśli missingRatio ≤ 5% → **usuń wiersze** z brakującymi wartościami w danej kolumnie (rmmissing).

```
for i = 1:width(data)
    col = data{:, i};
    missingRatio = sum(ismissing(col)) / height(data);
    if missingRatio > 0.05
        if isnumeric(col) || islogical(col)
            data{ismissing(col), i} = median(col, 'omitnan');
        elseif iscell(col)
            modeVal = mode(categorical(col));
            data{ismissing(col), i} = {char(modeVal)};
        else
            modeVal = mode(categorical(string(col)));
            data{ismissing(col), i} = char(modeVal);
        end
    end
```

```

else
    data = rmmissing(data, 'DataVariables',
data.Properties.VariableNames{i});
end
end

```

### 3.3 Kodowanie zmiennych kategoriycznych

- **Gender:** Female → 1, Male → 0.

```

data.Gender = string(data.Gender);
data.Gender(data.Gender == "Female") = "1";
data.Gender(data.Gender == "Male") = "0";
data.Gender = str2double(data.Gender);

```

- **Married:** Yes → 1, No → 0.

```

data.Married = string(data.Married);
data.Married(data.Married == "Yes") = "1";
data.Married(data.Married == "No") = "0";
data.Married = str2double(data.Married);

```

- **Dependents:** 3+ → 3 (konwersja do liczby).

```

data.Dependents = string(data.Dependents);
data.Dependents(data.Dependents == "3+") = "3";
data.Dependents = str2double(data.Dependents);

```

- **Education:** Graduate → 1, Not Graduate → 0.

```

data.Education = string(data.Education);
data.Education(data.Education == "Graduate") = "1";
data.Education(data.Education == "Not Graduate") = "0";
data.Education = str2double(data.Education);

```

- **Self\_Employed:** Yes → 1, No → 0.

```

data.Self_Employed = string(data.Self_Employed);
data.Self_Employed(data.Self_Employed == "Yes") = "1";
data.Self_Employed(data.Self_Employed == "No") = "0";
data.Self_Employed = str2double(data.Self_Employed);

```

- **Property\_Area:** Rural → 0, Semiurban → 1, Urban → 2

```

data.Property_Area = string(data.Property_Area);
data.Property_Area(data.Property_Area == "Rural") = "0";
data.Property_Area(data.Property_Area == "Semiurban") = "1";
data.Property_Area(data.Property_Area == "Urban") = "2";
data.Property_Area = str2double(data.Property_Area);

```

- **Loan\_Status (target):**  $N \rightarrow 0, Y \rightarrow 1$ .

```
data.Loan_Status = string(data.Loan_Status);
data.Loan_Status(data.Loan_Status == "N") = "0";
data.Loan_Status(data.Loan_Status == "Y") = "1";
data.Loan_Status = str2double(data.Loan_Status);
```

### 3.4 Normalizacja zmiennych numerycznych

- **Kolumny normalizowane:**

ApplicantIncome, CoapplicantIncome, LoanAmount, Loan\_Amount\_Term.

- **Metoda – Normalizacja min-max**

- **Zabezpieczenie:**

Jeśli wszystkie wartości w kolumnie są identyczne (np.  $\max = \min$ ), kolumna jest wypełniana zerami.

```
for i = 1:length(colsToNormalize)
    colName = colsToNormalize(i);
    colData = data.(colName);

    if any(ismissing(colData))
        warning("Kolumna %s nadal zawiera braki!", colName);
    end
    colMin = min(colData);
    colMax = max(colData);

    if colMax ~= colMin
        data.(colName) = (colData - colMin) / (colMax - colMin);
    else
        data.(colName) = zeros(size(colData));
    end
end
```

## 4. Projekt Sieci

### 4.1 Przygotowanie danych do sieci neuronowej

- **Dane wejściowe (P):**
  - Wszystkie kolumny oprócz Loan\_Status (cechy).
  - Konwersja tabeli do macierzy i transpozycja → wymiar [cechy x próbki].

```
P = table2array(data(:, 1:end-1))';
```

- **Dane wyjściowe (T):**
  - Kolumna Loan\_Status (target).
  - Transpozycja → wymiar [1 x próbki].

```
T = table2array(data(:, end))';
```

### 4.2 Architektura sieci

Sieć została utworzona przy użyciu funkcji newff, czyli Feedforward Neural Network (FFNN) klasyczna sieć neuronowa jednokierunkowa z pełnym połączeniem między warstwami.

Warstwa ukryta:

- Liczba neuronów: 30
- Funkcja aktywacji: logsig

Warstwa wyjściowa:

- Liczba neuronów: 1
- Funkcja aktywacji: purelin

Parametry uczenia:

- Współczynnik uczenia kontrolujący szybkość aktualizacji wag: 0.005
- Maksymalna liczba epok: 20
- Minimalny błąd, jeśli który zostanie osiągnięty, trening się kończy: 0.0005
- Algorytm uczący, domyślnie funkcja newff używa trainlm

Podziału danych:

- Zbiór treningowy: 80%
- Zbiór testowy: 10%
- Zbiór walidacyjny: 10%

Zbiór walidacyjny to dane niewidoczne dla modelu podczas uczenia. Został zastosowany, aby unikać przeuczenia sieci. Oczywiście nie jest on używany do aktualizacji wag.

## 4.3 Przebieg uczenia sieci

Opisane w poprzednim podrozdziale atrybuty sieci zostały zaimplementowane w kodzie znajdującym się poniżej:

```
net.trainParam.lr = 0.005;  
net.trainParam.epochs = 20;  
net.trainParam.goal = 0.0005;  
net.divideParam.trainRatio = 0.8;  
net.divideParam.valRatio = 0.1;  
net.divideParam.testRatio = 0.1;
```

Uczenie rozpoczyna się w linijce kodu:

```
[net, TR] = train(net, P, T);
```

Wynikami uczenia są:

**Training Results**  
Training finished: Met validation criterion 

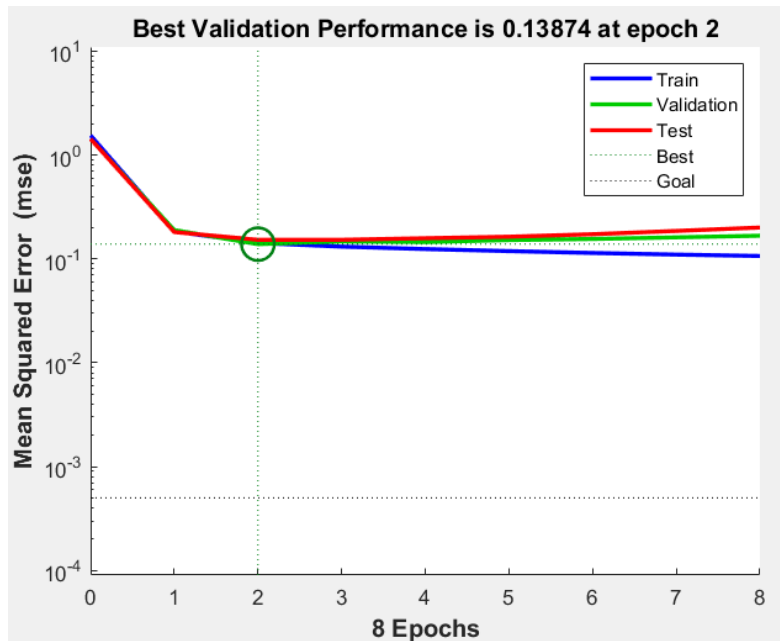
**Training Progress**

| Unit              | Initial Value | Stopped Value | Target Value |
|-------------------|---------------|---------------|--------------|
| Epoch             | 0             | 8             | 20           |
| Elapsed Time      | -             | 00:00:02      | -            |
| Performance       | 1.54          | 0.107         | 0.0005       |
| Gradient          | 4.38          | 0.0113        | 1e-07        |
| Mu                | 0.001         | 0.0001        | 1e+10        |
| Validation Checks | 0             | 6             | 6            |

**Training Algorithms**  
Data Division: Random dividerand  
Training: Levenberg-Marquardt trainlm  
Performance: Mean Squared Error mse  
Calculations: MEX

Z rysunku możemy odczytać, że trening został zatrzymany przez mechanizm early stopping czyli:

- sieć nie osiągnęła jeszcze docelowego błędu (Performance Target), ale
- błąd walidacyjny nie poprawiał się przez 6 kolejnych epok więc dalsze uczenie groziłoby przeuczeniem



Na początku błąd na wszystkich trzech zbiorach szybko spada co oznacza, że sieć uczy się poprawnie i uogólnia dobrze. Najlepszy błąd walidacyjny został osiągnięty w 2 epoche. Zaczynając od 3 epoki sieć zaczyna się przeuczać. Jest to klasyczny przypadek, gdy model zapamiętuje dane treningowe, ale traci zdolność generalizacji na nowych danych.

## 5. Analiza rezultatów i wnioski

### 5.1 Miary jakości

Pierwszym krokiem w analizie uzyskanych wyników jest zaokrąglenie odpowiedzi pozyskanych z sieci. Pozwala to na określenie do jakiej klasy zostały zakwalifikowane poszczególne obiekty. Kod, który za to odpowiada to:

```
Ptest = P(:, TR.testInd);
Ttest = T(:, TR.testInd);
Ytest = sim(net, Ptest);
Yclass = Ytest > 0.5;
Ttest = double(Ttest);
Yclass = double(Yclass);
```

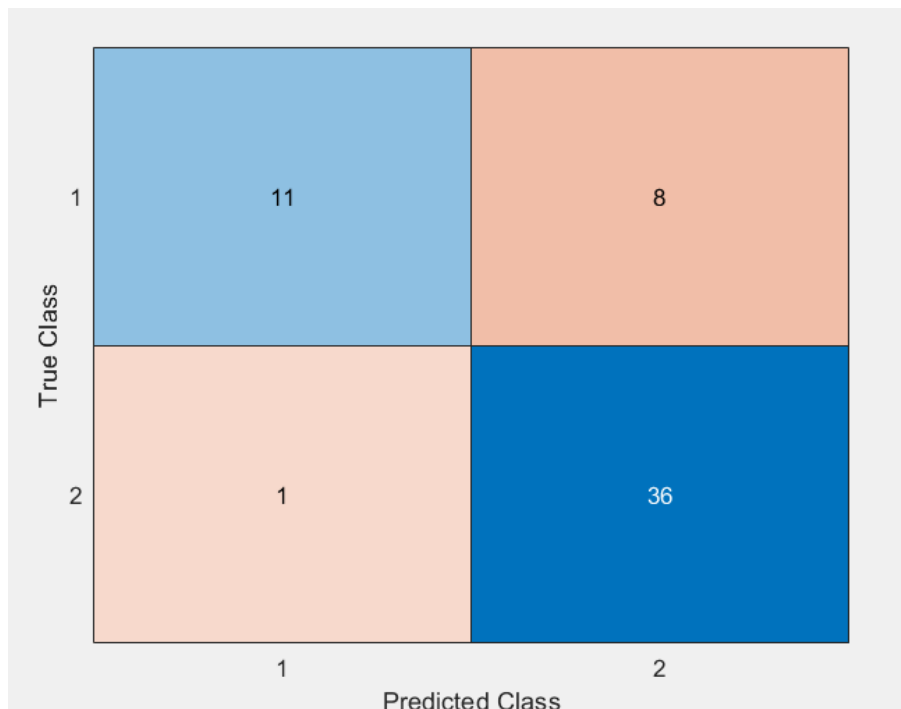
Metrykami, które zostały użyte do ocenienia miary jakości działania sieci są:

- Accuracy - odsetek wszystkich poprawnych klasyfikacji.
- Precision - dokładność przewidywań pozytywnych.
- Recall - czułość: jaki procent pozytywnych przypadków wykryto.

Wykorzystano także graficzną reprezentację w formie macierzy pomyłek. Kod odpowiedzialny za wyświetlenie macierzy pomyłek to:

```
cm = confusionmat(Ttest, Yclass);
TP = cm(2,2);
FP = cm(1,2);
FN = cm(2,1);
TN = cm(1,1);
confusionchart(cm);
```

Poniższy rysunek przedstawia macierz pomyłek. Można z niej odczytać, że 11 przypadków zostało dobrze rozpoznanych jako nieudzielenie kredytu i tylko jeden został błędnie rozpoznany. Można także odczytać z niej, że 36 przypadków zostało poprawnie rozpoznanych jako udzielenie kredytu a 8 błędnie jako nie udzielenie kredytu.



Macierz ta pozwala na określenie wartości:

- TN – true positive
- FP – false positive
- FN – false negative
- TN – true negative

Za pomocą tych wartości można obliczyć miary jakości accuracy, precision i recall. Kod który za to odpowiada:

```
accuracy = (TP + TN) / sum(cm(:));
```

```
precision = TP / (TP + FP);
```

```
recall = TP / (TP + FN);
```

```
fprintf('Accuracy: %.2f\n', accuracy*100);
```

```
fprintf('Precision: %.2f\n', precision*100);
```

```
fprintf('Recall: %.2f\n', recall*100);
```

Wynikiem pracy tego kodu jest:

```
Accuracy: 83.93
```

```
Precision: 81.82
```

```
Recall: 97.30
```

- Accuracy (dokładność): 83.93% - Oznacza, że model poprawnie sklasyfikował około 84% przypadków z zestawu testowego (czyli ile razy przewidywanie zgadza się z rzeczywistą etykietą). To dobry wynik, szczególnie jeśli dane są zrównoważone, choć dokładność samodzielnie nie zawsze jest najlepszą miarą w problemach z niezbalansowanymi klasami.
- Precision (precyzja): 81.82% - Spośród wszystkich wniosków, które model oznaczył jako zaakceptowane (klasa pozytywna), prawie 82% faktycznie było zaakceptowanych. Wysoka precyzja oznacza, że model rzadko „fałszywie” przyznaje kredyt (niskie FP).
- Recall (czułość): 97.30% - Model wykrywa aż 97.3% wszystkich faktycznie zaakceptowanych wniosków. Bardzo wysoka czułość oznacza, że model prawie nie pomija prawdziwych pozytywów (niskie FN). To ważne, bo oznacza, że rzadko odrzuca wnioski, które faktycznie powinny być zaakceptowane.

## 5.2 Wnioski

Projekt wykazał, że zastosowanie sieci neuronowej do klasyfikacji wniosków kredytowych pozwala na skuteczne wspomaganie decyzji bankowych, osiągając wysoką dokładność (około 84%) oraz bardzo wysoką czułość (97%), co oznacza minimalizowanie odrzucania wiarygodnych klientów. Model wykazał też dobrą precyzję (82%), co przekłada się na ograniczenie liczby fałszywych akceptacji, równoważąc ryzyko i efektywność działania. Kluczowym elementem sukcesu była staranna obróbka danych, obejmująca uzupełnianie braków i normalizację, podkreślając, jak ważna jest jakość danych dla działania algorytmu. Wyniki projektu potwierdzają potencjał automatyzacji procesów kredytowych w sektorze finansowym, zwiększając szybkość i obiektywność decyzji, choć wskazują też na potrzebę dalszej optymalizacji i uwzględnienia aspektów interpretowalności oraz zgodności z regulacjami, aby system mógł być odpowiedzialnie wdrożony w praktyce.